

TryHackMe

Learning Sources

[TryHackMe](#)

Notes

-Notes originally created in Sept 2025

01. HTTP in Detail

-Note: This is review. I already knew this.

HTTP

-Hypertext Transfer Protocol

-A set of rules developed in 1989-1991 for communicating with web servers for web page data transmission (html, assets, etc.)

HTTPS

-Hypertext Transfer Protocol Secure

-A secure version of HTTP via end-to-end encryption before data is sent plus verification server talking to is intended server

URL

-Uniform Resource Locator

-Used to request to a web server. Ex. `https://tryhackme.com:443/view-room?id=1#task3`

-Parts of URL:

- Scheme: what protocol to use. ex. `https`
- Host: domain for server to access. ex. `tryhackme.com`
- Port: port to use for connection. ex. `443`
- Path: path on server to access. ex. `view-room`
- Query Param: how to filter requested resource. ex. `id=1`
- Fragment: references a location in a page. ex. `#task3`

-URL may also require auth to access resource, such as username and password

Example Request

`POST /login HTTP/1.1`

Post to /login using HTTP ver 1.1

`Host: tryhackme.com`

from this host

`User-Agent: Mozilla/5.0 Firefox/87.0`

on this browser

`Referer: https://tryhackme.com/`

from this site

`Content-Type: application/x-www-form-urlencoded`

form data being sent

`username=thm&password=letmein`

form data

Example Response

`HTTP/1.1 200 OK`

Server: nginx/1.15.8
Date: Fri, 09 Apr 2021 13:34:03 GMT
Content-Type: text/html
Content-Length: 98

<html>
...
</html>

HTTP Methods

- GET – Get data from server
- POST – submit data to server (incl. create new record)
- PUT – Replace existing data fully
- PATCH – Replace existing data partially
- DELETE – Delete existing data

HTTP Status Codes

- 1xx – Info response (ex. request accepted). Not much used.
- 2xx – Request successful
- 3xx – Redirection
- 4xx – Client request error (ex. 404, not found)
- 5xx – Server error (ex. 501, not implemented)

Common HTTP Status Codes

200 – OK	201 – Created	301 – Moved Permanently	302 – Found
400 – Bad Request	401 – Not Authorized	403 – Forbidden	
405 – Method Not Allowed	404 – Not Found	500 – Internal Server Error	
503 – Service Unavailable			

Common Request Headers

- Metadata about request. Not required by HTTP, but generally need for most requests.
- Host – For if server hosts multiple websites, so knows which one to request to
- User-Agent – browser type (ex. Firefox) and version num, to help with formatting, etc.
- Content-Length – how much data to expect so server can ensure no data missing
- Accept-Encoding – what types of compression to expect so data can be made smaller for sending
- Cookie – cookie data sent to server on each request (ex. auth cookie)

Common Response Headers

- Metadata sent back as part of response.
- Set-Cookie – sets client side cookie
- Cache-Control – how long to store content before browser requests again
- Content-Type – what type of data is being sent to client (ex. HTML, CSS, PDF, etc.)
- Content-Encoding – what types of compression to expect so data can be made smaller for sending

Cookies

- Client-side stored data set via *Set-Cookie* header by server. Each request user makes after cookie set sends back cookie to server. Used to verify auth, etc.
- A way to store temp client-side data while still allowing HTTP to be stateless (not track previous requests)
- Often used for website auth, such as a JWT token, which handles both authentication and authorization.
- Can view cookies via Browser Tools → Network → [request] → Cookies

02. Burp Suite: The Basics

Burp Suite 101

- A Java-based framework designed to act as a solution for comprehensive web/mobile-application pen testing. Industry standard for hands on testing of web apps, including APIs.
- Burp Suite captures and enables manipulation of all HTTP/HTTPS traffic between a browser and web server
- Requests/responses can be intercepted and routed to various Burp components for analysis/manipulation. A proxy on both ends.
- Burp Suite exists in community, Professional, and Enterprise additions, where Enterprise features constant scanning of a server.

Features

- Proxy – for interception and manipulation of requests and responses
- Repeater – for capturing, modifying, and re-sending the same request multiple times. Especially useful for trial-and-error payloads (ex. *SQLi*)
- Intruder – for spraying endpoints w/ requests. Often used for brute-forcing or fuzzing endpoints.
- Decoder – for decoding captured info or encoding payloads before sending to target.
- Comparer – for comparing two pieces of data on word or byte level.
- Sequencer – for assessing randomness of tokens (ex. session cookies) and other pseudo-random data. Useful to expose bad encryption.
- Also allows extensions to be added, for greater functionality. Many are available in the *BApp* store.

Installation

- Burp is installed by default on Kali Linux. For other Linux and Win, download install file / script from official

The Dashboard

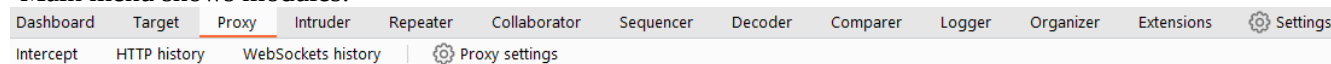
- The dashboard includes 4x main panes: Tasks, Event Log, Issue Activity, and Advisory
- Tasks – Allows to define a BG task to perform while use application (ex. Live Passive Crawl from Proxy)
- Event Log – Logs actions performed by Burp (ex. start proxy) and connections made through Burp

-Issue Activity – Burp Pro only. Displays vulnerabilities detected by scanner, ranked by severity & filterable based on level

-Advisory – Provides detailed info about detected vulnerabilities, includes references & suggested remedies

Navigation

-Main menu shows modules:



-Each module provides its own interface

-Can quick navigate to modules via *ctrl + shift + [char]*, with some *char* being *D* (Dashboard), *T* (Target), *P* (Proxy), *I* (Intruder), and *R* (Repeater)

Settings

-Before using Burp, should set up Global and Project settings via *Settings Icon → Tools → Proxy* and *Settings Icon → Project → Scope*

-If do not set *Scope* to be domain, etc. limited, it will scan all HTTP requests sent on Burp's browser, which can violate privacy or create a lot of noise

-Can search settings to find/update a specific setting

-Can also use settings to manage uploaded Client-Side TLS certificates, where we override these on a per-project basis

Burp Proxy

-A fundamental tool w/i Burp Suite to capture requests and responses between client / server. Intercepted traffic can then be mutated before being sent, sent to other tools for further processing before being sent, or explicitly allowed to continue to the destination.

-To turn on interception, toggle *Intercept On* in main Proxy *Intercept* tab. With *Intercept On*, all requests stop and Burp and wait to continue until either *Forward* (send it on), *Drop* (stop request), or *Send* to some other tool, like *Repeater*.

-Interception allows:

- Taking control of web requests for complete tester control over web traffic
- Capture and logging for later analysis and review
- WebSocket support for capturing and logging websocket comms
- Logs & history for review in HTTP History and Web Socket History tool sub-tabs

-Note, that by default Burp Proxy does not intercept responses so if this is desired, will need to enable under Proxy settings

-Can also set “match and replace” regex details in the proxy section to auto-match and replace parts of requests/responses. Useful for manipulating user agents, cookies, etc.

-Once Proxy has captured an HTTP comm, it can send to other Burp tools, such as Repeater, for additional manipulation

Burp Target

-Tool to provide more control over the testing scope. Contains three main sub-tabs: Site Map, Issues, & Scope

Settings

-Site Map – allows to map out targeted web apps in a tree structure. Each site (ex. *google.com*, *tryhackme.com*) is it's own root. When click root, can see all paths, assets, etc. in site and explore them.

-Issues – Shows a full list of vulnerabilities the scanner looks for for each target. Useful for referencing what vulnerabilities are defined and what they mean.

-Scope Settings – Allows setting of the Target scope to define what hosts and URLs to target for scanning by Burp

-Target thus mostly used for defining Burp scanning scope, exploring issue definitions, and for exploring web domains visited

Burp Browser

-Burp Suite includes a built-in Chromium browser that is per-configured to use Proxy w/o any additional setup required

-To open, go to *Proxy*, then click *Open Browser*

-Proxy will auto-capture ALL HTTP/WebSocket sent in this Chromium

Scoping & Targeting

-*Target* captures all communications sent via the Burp Chromium browser. We likely only want to capture and log some traffic to some domains, though.

-This is done via going to the *Target* tool, then clicking on a site, then adding to scope via clicking *Add to scope*. We can then see the site added to *Target* → *Scope*.

-With scopes added, we can filter noise, by checking things such as *show only in-scope items* in *Proxy* → *HTTP History* and other tools. Additionally, *Scanner*, *Intruder*, and *Crawler* tools only hit in scope hosts, which allows prevention of hammering sites not intended to touch.

-To only capture/log in-scope sites by *Proxy*, as opposed to all requests/responses, we can set *And URL is in target scope* to request and response interception rules in *Proxy Settings*. If want to see all sites in *HTTP History* again we can toggle *Filter on* to *Filter off*

Proxying HTTPS

-TLS -Transport Security Layer. A crypto protocol for securing traffic for communication on a network. Provides auth via use of certs.

-When intercepting HTTP traffic, if site uses TLS, as in HTTPS requests/responses, may receive error saying Certificate Authority is not authorized to secure the connection. This happens as browser does not trust the cert presented by Burp Suite.

-To fix, need to manually add the missing cert to browsers list of trusted certs. For local burp do this via:

1. Navigate to *http://burp/cert* and save cert
2. Add to cert to trusted certs in browser via browser settings (ex. *certificates* in *about:preferences* in FF)

Example Attack

-After booting TryHackMe's AttackBox and the Burp Suite installed it, we can access *http://[LAB_MACHINE_IP]/ticket/* site via Proxy Browser on AttackBox Burp.

-For example attack, we have a form on *ticket* path. The client-side form validation prevents us from entering restricted data, such as `<script>...</script>`, but this is okay, as we can turn *Intercept on* in *Proxy*, then enter valid data into form, the capture request, then modify data being submitted by form to include `<script>...</script>` as form data anyways, such as how we may do so in an XSS attack.